

Analyse Stratégique de l'Usage d'IA Locale

(Gemma 4)

Optimisation de la fenêtre de contexte pour les projets de grande envergure

Projet Farmstar • 2026

Table des matières

1. Contexte Technique en 2026	3
2. Cas d'Usage	3
3. Orchestration par Model Context Protocol (MCP)	4
3.1. Concept Fondamental : Du RAG Statique à l'Agent Actif	4
3.2. Architecture de Déploiement	4
3.3. Catalogue d'Outils Spécifiques (Farmstar Tools)	4
3.3.1. Outils de Navigation et Structure (Project File Tools)	4
3.3.2. Outils d'Accès au Code (File Analysis Tools)	4
3.3.3. Outils de Recherche (Search Tools)	5
3.3.4. Outils de Partage de Connaissances (Skills)	5
3.4. Sécurité et Gouvernance	5
3.5. Orchestration Multi-Agent : Sortir des Silos	5
3.5.1. Le Défi de l'Implémentation Actuelle	5
3.5.2. Solution : Orchestration par Graphe d'État (LangGraph)	5
3.6. Architecture Complète :	5
4. Le Concept de Context Engineering	6
4.1. Les Stratégies d'Optimisation du Contexte	6
4.1.1. Le « Chunking » Sémantique (Segmentation Logique)	6
4.1.2. L'Enrichissement par Métadonnées (Metadata Injection)	6
4.1.3. La Recherche Hybride (Dense & Sparse Retrieval)	6
4.1.4. Le Re-ranking (Filtrage de Haute Précision)	7
4.1.5. La « Skeletonization » Dynamique (Architecture à la demande)	7
4.2. L'Ingénierie des Instructions (Prompting)	7
4.3. Synthèse : Passer du Brut à l'Ingénierie	7
5. Stratégie d'Alimentation du RAG (Ingestion des Données)	8
5.1. Les Sources de Données (Le « Quoi »)	8
5.2. Le Pipeline de Transformation (Le « Comment »)	8
6. Implémentation Technique et Connectivité	9
6.1. Intégration IDE : L'Assistant de Flux	9

6.2. Interface Centralisée : Le Cerveau Partagé	9
6.3. Revue de Code et CI/CD	9
7. Contraintes Opérationnelles	9
8. Glossaire Technique	11

1. Contexte Technique en 2026

L'arrivée de Gemma 4 (2.3B à 31B) marque un tournant pour le développement « on-premise ». Avec une fenêtre de contexte native de 128k tokens, ces modèles permettent d'analyser des structures de code complexes.

Toutefois, le projet Farmstar nécessite évidemment bien plus que 128k tokens de code source, d'où la nécessité d'adopter des stratégies de Context Engineering pour maximiser l'efficacité de l'IA locale sans saturer la mémoire.

2. Cas d'Usage

Les 5 axes d'implémentation

Les points suivants couvrent les intégrations prioritaires pour Farmstar en 2026, constituant un véritable pipeline IA du développement dès la conception jusqu'à la validation.

1. Assistant de Code

L'IA intervient comme un « binôme » de programmation en temps réel. Elle ne se contente pas de compléter la syntaxe, mais suggère des implémentations basées sur les patterns spécifiques de Farmstar (Clean Code).

2. Workflow Ticket Jira

L'IA analyse le ticket, identifie les modules impactés (Front/Back), génère la spécification et propose un plan d'action d'implémentation.

3. Revue de Code

L'IA agit comme un premier filtre lors des Pull Requests. Elle vérifie la conformité du code par rapport aux standards de l'entreprise avant que le développeur humain n'intervienne.

4. Génération de Tests

L'IA automatise la tâche chronophage de l'écriture des tests unitaires et d'intégration (JUnit/Mockito), en simulant des cas aux limites souvent oubliés.

5. Aide au « Retro-Engineering »

L'IA sert de guide interactif pour analyser un code existant, reconstituer son architecture, identifier ses dépendances et comprendre sa logique métier sans documentation complète.

3. Orchestration par Model Context Protocol (MCP)

L'architecture RAG décrite précédemment permet de fournir des faits à l'IA. Cependant, pour transformer Gemma 4 en un véritable assistant opérationnel pour Farmstar, nous devons passer d'une consommation passive de données à une capacité d'action dynamique. C'est ici qu'intervient le Model Context Protocol (MCP).

3.1. Concept Fondamental : Du RAG Statique à l'Agent Actif

Le MCP permet de standardiser la manière dont le modèle interagit avec des outils tiers sans nécessiter de développements spécifiques pour chaque interface (Open WebUI, IDE, CLI).

Concept clé : Tool Calling

Contrairement au RAG où le système injecte le contexte **avant** la question, le MCP permet au modèle de décider **pendant** son raisonnement s'il a besoin d'un outil (ex: `read_jira_ticket`, `exec_sql_query`) pour compléter sa tâche.

Synergie RAG + MCP

Le RAG fournit la connaissance (recherche sémantique globale), tandis que le MCP fournit la capacité d'action (outils métier). Ensemble, ils permettent à Gemma 4 de ne plus seulement « répondre », mais de « résoudre ».

3.2. Architecture de Déploiement

Pour Farmstar, l'implémentation du MCP suivra une structure hybride adaptée aux contraintes de sécurité et de performance:

- ▶ Transport stdio (Local) : Utilisé par les plugins d'IDE (Continue/Tabby). Le serveur MCP tourne sur le poste du développeur, permettant un accès direct au système de fichiers local et aux scripts de build.
- ▶ Transport SSE/HTTP (Centralisé) : Utilisé par l'interface Open WebUI. Les serveurs MCP sont hébergés sur l'infrastructure serveur, permettant d'interroger les API centralisées (Jira, GitLab, Qdrant) de manière unifiée.
- ▶ Sandboxing : Chaque outil critique (exécution de code, accès DB) doit s'exécuter dans un environnement isolé (Sandbox) pour éviter toute action destructive sur le projet.

3.3. Catalogue d'Outils Spécifiques (Farmstar Tools)

L'implémentation actuelle du serveur MCP Farmstar expose les outils suivants, organisés en quatre familles fonctionnelles :

3.3.1. Outils de Navigation et Structure (Project File Tools)

Ces outils permettent à l'IA de comprendre l'organisation générale du monorepo sans être submergée par des milliers de fichiers.

- `list_project_files()` : Liste les fichiers clés du projet (Python, YAML, Markdown) pour orienter les recherches ultérieures.
- `list_project_structure(depth: int)` : Fournit une arborescence simplifiée des dossiers jusqu'à une profondeur donnée, en excluant les répertoires non pertinents (`.git`, `.venv`, `target`, etc.).
- `list_files_in_directory(dir_path: str)` : Liste le contenu d'un dossier spécifique, utile après localisation d'un module intéressant.

3.3.2. Outils d'Accès au Code (File Analysis Tools)

Ces outils permettent à l'IA de consulter le code source en détail.

- `read_project_file(file_path: str)` : Lit le contenu complet d'un fichier du monorepo avec vérification de sécurité (confinement à l'intérieur du répertoire root).

- **get_file_summary(file_path: str)** : Fournit un résumé intelligent du contenu d'un fichier en utilisant le moteur RAG, utile pour les fichiers volumineux.

3.3.3. Outils de Recherche (Search Tools)

Deux modes de recherche complémentaires pour localiser du code :

- **search_farmstar_code(query: str)** : Recherche **sémantique** (conceptuelle) dans le codebase. À utiliser pour comprendre une logique métier globale quand on ne connaît pas le nom exact des fichiers (ex: « gestion de la biomasse », « flux de notification »).
- **search_by_regex(pattern: str, file_extension: str)** : Recherche **technique** (exacte) utilisant les expressions régulières. Pour trouver des correspondances précises (ex: `public class .*Controller`).

3.3.4. Outils de Partage de Connaissances (Skills)

- **list_and_read_skill(skill_name: str)** : Liste et consulte les skills (fichiers Markdown) disponibles. Permet à l'IA de se référer à des guides ou des bonnes pratiques prédéfinis (ex: architecture, patterns de codage).

3.4. Sécurité et Gouvernance

L'usage du MCP impose une gestion fine des droits d'accès. Un orchestrateur de contexte devra filtrer les outils disponibles en fonction du rôle de l'utilisateur (Développeur vs Tech Lead) et journaliser chaque appel d'outil pour assurer la traçabilité des modifications effectuées par l'IA.

3.5. Orchestration Multi-Agent : Sortir des Silos

3.5.1. Le Défi de l'Implémentation Actuelle

L'implémentation actuelle via Open WebUI révèle une limite structurelle : les agents (Jira, Farmstar, Code) fonctionnent en **silos**. Un agent « Jira » ne peut pas transmettre de contexte directement à l'agent « Analyse de Code » sans intervention humaine.

Dans un workflow complexe (ex: « Résoudre le ticket FS-402 »), l'IA doit :

1. Lire le ticket (Agent Jira).
2. Analyser les fichiers sources impactés (Agent Farmstar).
3. Proposer un correctif et vérifier les dépendances (Agent Code).

Actuellement, l'utilisateur doit effectuer manuellement le « copier-coller » du contexte entre chaque agent. Cela engendre une perte de cohérence et augmente le risque d'hallucination.

3.5.2. Solution : Orchestration par Graphe d'État (LangGraph)

Pour que le model devienne un véritable assistant autonome, nous devons déporter la logique d'orchestration de l'interface (Open WebUI) vers une **couche de logique métier (Middleware)**.

Orchestration par Graphe d'État

L'utilisation de frameworks comme **LangGraph** permet de définir un « Cerveau Central » (Supervisor) qui :

- Détient l'état global de la tâche (State).
- Appelle dynamiquement les outils MCP nécessaires.
- Oriente le flux de travail entre les différents « nœuds » de compétences sans perte de contexte.

3.6. Architecture Complète :

L'architecture cible pour Farmstar repose sur la synergie de trois composantes :

- **RAG** : La mémoire sémantique (Connaissance globale et recherche contextuelle).
- **MCP** : Le bras opérationnel (Capacité d'action sur les outils métier Farmstar).
- **LangGraph** : Le chef d'orchestre (Planification, coordination et état global des tâches).

4. Le Concept de Context Engineering

Le Context Engineering est l'ensemble des techniques permettant de transformer une masse de données brutes (le code source de Farmstar) en une sélection d'informations structurées et pertinentes pour le modèle de langage.

À l'échelle de plusieurs millions de tokens, le modèle ne peut pas « voir » tout le projet. L'ingénierie de contexte consiste donc à construire un « **résumé intelligent** » et dynamique du projet.

Pourquoi est-ce indispensable pour Farmstar ?

Le projet Farmstar dépasse la capacité de mémoire immédiate de Gemma 4. Sans ingénierie, l'IA souffre de deux maux majeurs :

- **Saturation** : Trop d'informations inutiles (bruit) masquent les informations vitales.
- **Dilution (Lost in the Middle)** : Plus le contexte est long, plus le modèle a du mal à faire des liens précis entre des éléments éloignés.

4.1. Les Stratégies d'Optimisation du Contexte

Le défi pour Farmstar n'est pas seulement de trouver l'information, mais de la compresser et de la prioriser pour que Gemma 4 reste performante malgré la masse de données.

4.1.1. Le « Chunking » Sémantique (Segmentation Logique)

Au lieu d'un découpage arbitraire par nombre de caractères, le RAG utilise une segmentation basée sur la structure du code.

- **Principe** : Utilisation de parseurs (ex: Tree-sitter) pour isoler des unités logiques complètes.

Exemple : Plutôt que de couper le fichier `BiomasseService.java` tous les 1000 caractères, le RAG extrait la méthode `public double calculerIndiceVegetation(...)` dans son intégralité, du début de la signature jusqu'à l'accolade fermante.

4.1.2. L'Enrichissement par Métadonnées (Metadata Injection)

Chaque fragment récupéré par le RAG est « augmenté » par un préfixe textuel avant d'être soumis à l'IA.

- **Principe** : On injecte des informations de contexte global (chemin, branche, module) en haut du fragment.

Exemple de préfixe injecté :

```
[FILE: agri-frontend/src/components/MapScale.ts | MODULE: geo-view | BRANCH: develop]
export const updateScale = (zoom: number) => { ... }
```

4.1.3. La Recherche Hybride (Dense & Sparse Retrieval)

La recherche hybride combine deux méthodes de récupération pour compenser leurs limites respectives. Elle assure que l'IA dispose à la fois d'une compréhension conceptuelle et d'une précision lexicale.

Recherche Dense (Vectorielle) : Cette méthode convertit le texte en coordonnées mathématiques (embeddings). Elle permet de saisir l'intention derrière une question.

Avantage : Elle gère les synonymes et les concepts proches. Une requête sur la « santé des cultures » pourra identifier des segments traitant de « l'indice de végétation ».

Recherche Sparse (Mots-clés / BM25) : C'est la méthode de recherche traditionnelle par indexation de termes précis.

Avantage : Elle est indispensable pour le code source. Elle garantit que les noms techniques exacts (ex: `FSAgriCultureValidator`) soient trouvés, là où une recherche vectorielle pourrait proposer un validateur générique par simple ressemblance sémantique.

Application au projet Farmstar

Le monorepo contient des milliers de classes aux noms très spécifiques. La recherche hybride permet de répondre à une question métier floue (« Comment traiter les anomalies de surface ? ») tout en restant capable de pointer immédiatement un fichier technique précis si son nom est mentionné.

4.1.4. Le Re-ranking (Filtrage de Haute Précision)

Le processus de récupération initial (RAG classique) extrait souvent une cinquantaine de fragments pour maximiser les chances de trouver l'information. Cependant, injecter 50 fragments dans le modèle génère du bruit et réduit la précision. Le Re-ranker intervient comme un second filtre expert.

Le fonctionnement : Contrairement à la recherche initiale qui est rapide mais superficielle, le Re-ranker utilise un modèle dit « Cross-Encoder ». Il analyse la relation directe entre la question posée et chaque fragment trouvé pour leur attribuer un score de pertinence réel.

L'utilité : Il permet de classer les résultats du plus pertinent au moins pertinent.

Lutter contre le phénomène « Lost in the Middle »

Les modèles de langage, y compris Gemma 4, perdent en attention sur les informations situées au centre d'un contexte volumineux. Le Re-ranking garantit que les trois fragments de code les plus critiques sont placés au sommet du contexte, là où le modèle exerce sa capacité de raisonnement maximale.

4.1.5. La « Skeletonization » Dynamique (Architecture à la demande)

Utilisée pour donner une « carte » à l'IA sans saturer sa mémoire.

- **Principe** : Injection des signatures sans le corps des fonctions.

Exemple : Pour comprendre une dépendance, on fournit le code complet du service appelant, mais seulement la signature du client appelé :

```
public class SatelliteClient {
    public Image getImageByCoord(double lat, double lon); // Corps masqué
}
```

4.2. L'Ingénierie des Instructions (Prompting)

Le Context Engineering inclut également la structuration des consignes pour garantir la fiabilité des réponses :

- ▶ ***Balisage (Tagging) *** Utilisation de balises structurées (XML/Markdown) pour délimiter les fichiers et aider l'IA à se repérer géographiquement dans le monorepo.
- ▶ ***Ancrage et Traçabilité (Source Grounding) *** Obligation faite au modèle de citer les fichiers sources utilisés pour chaque affirmation, permettant un audit rapide et limitant les hallucinations.
- ▶ ***Few-Shot Contextualisation *** Injection d'exemples de "résultats types" (ex: tests unitaires validés) pour aligner la génération sur les standards de qualité de Farmstar.

4.3. Synthèse : Passer du Brut à l'Ingénierie

Méthode	Donnée Brute	Ingénierie de Contexte
Volume	Tout le code source (Millions)	Sélection ciblée (Kilos)
Précision	Risque élevé d'hallucinations	Réponses basées sur des faits extraits

Performance	Lenteur (sature la VRAM)	Inférence fluide et rapide
-------------	--------------------------	----------------------------

5. Stratégie d’Alimentation du RAG (Ingestion des Données)

Une base vectorielle n’est pertinente que si la donnée qui l’alimente est qualifiée. Pour un écosystème de l’envergure de Farmstar, l’indexation ne se limite pas à un simple « copier-coller » du code. On met en place un véritable pipeline d’ingestion de connaissances.

5.1. Les Sources de Données (Le « Quoi »)

Pour obtenir une IA capable de faire le lien entre un besoin métier et une fonction technique, le RAG doit agréger trois typologies de documents :

- **Le Code Source Structuré (Java, TypeScript, SQL)** : Il ne doit pas être ingéré comme du texte brut. Il est « parsé » (souvent via un Abstract Syntax Tree) pour que la base de données comprenne où commence et où s’arrête une méthode ou une classe.
- **La Documentation Technique (Markdown, Swagger)** : Les fichiers README et les contrats d’API (JSON/YAML). Ils sont cruciaux car ils expliquent les « règles » du projet que le code seul ne décrit pas toujours.
- **Le Contexte Métier et Fonctionnel (PDF, Jira)** : Les spécifications (PDF/Word) et l’historique des tickets Jira. C’est cette couche qui permet à l’IA de comprendre le **pourquoi** d’une implémentation.

5.2. Le Pipeline de Transformation (Le « Comment »)

On ne « jette » pas un PDF de 50 pages ou une classe Java de 2000 lignes directement dans la base. Avant de devenir un vecteur mathématique, la donnée subit trois étapes :

Étape de l’ETL	Action sur la donnée	Exemple concret
1. Nettoyage	Exclusion systématique du bruit : fichiers compilés (.class), dossiers de dépendances, ou logs applicatifs.	Le RAG ignore automatiquement le dossier node_modules/.
2. Chunking Intelligent	Le code est découpé par « bloc logique ». Les PDF/Docs métiers sont découpés avec un chevauchement (overlap) de 15%.	Un chunk = la méthode Java entière calculBiomasse().
3. Enrichissement (Métadonnées)	On attache des « tags » invisibles à chaque vecteur pour forcer la recherche dans un périmètre précis.	Ajout des tags [Module: fs-core] et [Type: Spec].

Le Risque du "Garbage In, Garbage Out"

Si le RAG ingère de vieux documents PDF obsolètes ou du code mort, l’IA les utilisera comme source de vérité. L’indexation d’un projet comme Farmstar demande une gouvernance stricte de la donnée : l’index vectoriel doit être synchronisé avec une branche pré-déterminée et mis à jour régulièrement.

6. Implémentation Technique et Connectivité

La performance de Gemma 4 dépend de la « proximité » de l'outil avec le code source. On distingue deux modes de connexion principaux :

6.1. Intégration IDE : L'Assistant de Flux

Pour les cas d'usage nécessitant une réactivité instantanée (**Assistant de code, Génération de tests**), l'utilisation de plugins est impérative.

- **Outils types : Continue, Tabby.**
- **Mécanisme d'indexation :** Ils créent un **index vectoriel local** (via une DB légère comme LanceDB) directement sur le poste du développeur.
- **Avantage :** Le plugin « voit » ce que l'on tape et indexe les fichiers ouverts en temps réel pour un RAG de proximité sans latence (la base locale est mise à jour à chaque sauvegarde de fichier).

Bénéfices de l'indexation locale

- ✓ Latence minimale (< 200ms)
- ✓ Confidentialité des données (rien n'est envoyé vers le cloud)
- ✓ Adaptation en temps réel aux modifications du code

6.2. Interface Centralisée : Le Cerveau Partagé

Pour les tâches de plus haut niveau (**Workflow Jira, Aide à l'onboarding**), une interface Web (type **Open WebUI** ou **AnythingLLM**) serait à privilégier.

- **Le problème du contexte persistant :** On ne peut pas copier-coller des millions de lignes à chaque prompt.
- **La solution :** Coupler l'interface à une **base vectorielle centralisée** (ex: **Qdrant**) sur le serveur. Le modèle ne connaît pas le projet de mémoire, mais l'interface cherche les morceaux de code pertinents dans la base à chaque question de manière invisible. Il serait toujours possible de passer du contexte via document ou copié-collé classique pour les questions très spécifiques.

Avantage du RAG centralisé

Une base vectorielle unique garantit que tous les développeurs interrogent une source de vérité commune. Cela évite les divergences d'interprétation et assure une cohérence métier maximale.

6.3. Revue de Code et CI/CD

Pour la revue, le modèle doit être intégré dans GitLab.

- **Fonctionnement :** Un bot intercepte la Pull Request, récupère le « diff » (les modifications) et interroge Gemma 4 en fournissant ce différentiel et les règles de l'équipe.

7. Contraintes Opérationnelles

Ressources Matérielles

Le déploiement d'un modèle comme GEMMA 4 (8B à 31B) requiert une infrastructure GPU dimensionnée. Un minimum de **24Go de VRAM** (ex: RTX 3090 / 4090) est exigé pour garantir une inférence fluide en précision quantifiée, particulièrement avec de grands contextes.

Tableau de dimensionnement recommandé

Modèle	VRAM Min	Configuration Idéale	----	-----	-----	Gemma 2B	4 GB	RTX 3060 / RTX 4060
	Gemma 7B	8-12 GB	RTX 4070 / RTX 3080		Gemma 13B	16-20 GB	RTX 4080 / RTX 3090	Gemma 31B
	24+ GB	RTX 6000 / Multi-GPU						

8. Glossaire Technique

Terme	Définition
LLM	Large Language Model. Modèle de langage de grande taille (ex: Gemma 4) capable de comprendre et générer du texte ou du code.
RAG	Retrieval-Augmented Generation. Architecture qui permet à l'IA de consulter une base de connaissances externe avant de répondre.
Token	Unité de base traitée par l'IA. 1000 tokens correspondent environ à 750 mots. C'est la mesure de la « mémoire » immédiate du modèle.
Embedding	Représentation mathématique d'un texte sous forme de vecteur. Deux textes ayant un sens proche auront des vecteurs proches.
VRAM	Mémoire vidéo du GPU. Elle détermine la taille du modèle et la longueur du contexte que l'on peut charger localement.
Inférence	Processus de génération d'une réponse par le modèle à partir d'une requête donnée.
Chunking	Action de découper un document volumineux en segments plus petits pour faciliter leur indexation et leur recherche.
On-Premise	Installation logicielle sur les serveurs physiques de l'entreprise, garantissant une souveraineté totale des données.
BM25	Algorithme de recherche textuelle basé sur la fréquence des mots, utilisé pour la recherche de termes techniques précis.
Cross-Encoder	Modèle spécialisé dans le calcul de similarité entre deux textes. Utilisé en Re-ranking pour scorer la pertinence.
Tree-Sitter	Parser générique permettant d'analyser le code source et de construire un arbre syntaxique abstrait (AST).
Model Context Protocol (MCP)	Standard open-source (initié fin 2024) permettant de standardiser la communication entre les LLMs et des sources de données ou outils externes, remplaçant les intégrations API codées en dur.
Tool Calling	Mécanisme par lequel l'IA génère une commande structurée pour solliciter un outil externe via MCP, au lieu de générer du texte libre.
Agent Orchestrateur	IA capable de coordonner plusieurs outils et sources de données en temps réel pour accomplir une tâche complexe, dépassant la simple génération de texte.
Semantic Routing (Routage Sémantique)	Processus par lequel le système comprend l'intention de la requête de l'utilisateur et décide de la stratégie de recherche ou d'action à adopter (ex: RAG documentaire vs RAG code).

Sandbox (Bac à Sable)

Environnement d'exécution sécurisé dans lequel l'IA peut tester du code ou exécuter des commandes sans risque pour le système hôte. Utilisé pour valider les générateurs de code avant de les intégrer au projet.

 Mise à jour continue du glossaire

Ce glossaire est évolutif. À mesure que de nouvelles briques technologiques (comme les agents autonomes, MCP) seront intégrées au projet Farmstar, les définitions seront complétées pour assurer un niveau de compréhension optimal au sein de l'équipe.

Pour approfondir

Consultez la documentation officielle de Gemma : <https://ai.google.dev/gemma>

Plus d'infos sur Qdrant : <https://qdrant.tech>

Explorer le Model Context Protocol : <https://modelcontextprotocol.io>